

Taller de Testing en Java: Arquitectura y Verificación

Este documento es una guía práctica. No solo aprenderemos a escribir tests, sino a entender la arquitectura de software profesional. Antes de escribir una sola línea de código de prueba, debemos entender qué estamos construyendo.

Módulo 1: Arquitectura y Diseño del Sistema

Antes de probar, necesitamos entender el mapa de nuestra aplicación. No escribimos código "donde caiga"; lo organizamos en capas.

1.1. El Patrón Repositorio (The Repository Pattern)

Muchos estudiantes se preguntan: *"¿Por qué no guardamos los datos directamente en el Controlador o en el Servicio?"*.

Imagina que eres el gerente de una biblioteca inmensa. Si un cliente te pide "El Quijote", tú no corres personalmente por los pasillos buscando el libro. Tú le preguntas al **Bibliotecario**.

- Tú (Lógica de Negocio) solo quieres el libro.
- No te importa si el libro está en la estantería A, en el sótano, o si el bibliotecario tuvo que pedirlo a otra sucursal.
- El Bibliotecario (Repositorio) abstrae los detalles del almacenamiento.

Definición Técnica:

El Patrón Repositorio es una capa de abstracción entre la lógica de negocio (tu aplicación) y la capa de acceso a datos (Base de Datos, Archivos, Memoria).

¿Por qué lo usamos?

1. **Desacoplamiento:** Si mañana cambiamos la base de datos de MySQL a MongoDB, o a una lista en memoria (como haremos hoy), tu lógica de negocio **no cambia**. El "Bibliotecario" se encarga del cambio.
2. **Testabilidad:** Es la razón principal por la que estamos aquí. Es muy difícil probar una base de datos real. Es muy fácil "fingir" (mockear) a un bibliotecario.

1.2. El Desafío: Tu Misión (Definición de la API)

Tienes el encargo de construir el backend para la gestión de socios de un gimnasio. Debes implementar una API REST utilizando **Javalin**.

Arquitectura Requerida:

La aplicación debe tener 3 capas claras:

1. **Controlador (SociosController):** Habla HTTP (Javalin). Recibe peticiones y devuelve JSON.

2. **Servicio (ServicioSocios):** Habla Java. Contiene las reglas del negocio (validaciones).
3. **Repositorio (RepositorioSocios):** Habla Datos. Guarda y recupera socios.

Requisitos de la API (El Contrato):

Tu sistema debe cumplir estrictamente con estas especificaciones:

Endpoint 1: Registrar Socio

- **Método:** POST
- **Ruta:** /socios
- **Cuerpo (JSON):** { "nombre": "...", "email": "..." }
- **Reglas de Negocio (Servicio):**
 1. El nombre es obligatorio (no nulo, no vacío).
 2. El email debe ser válido (contener '@').
 3. El email debe ser único. No se puede registrar dos veces el mismo correo.
- **Respuestas HTTP:**
 - 201 Created: Si todo va bien. Devuelve el socio creado con su ID.
 - 400 Bad Request: Si falla alguna validación. Devuelve un JSON con el error.

Endpoint 2: Listar Socios

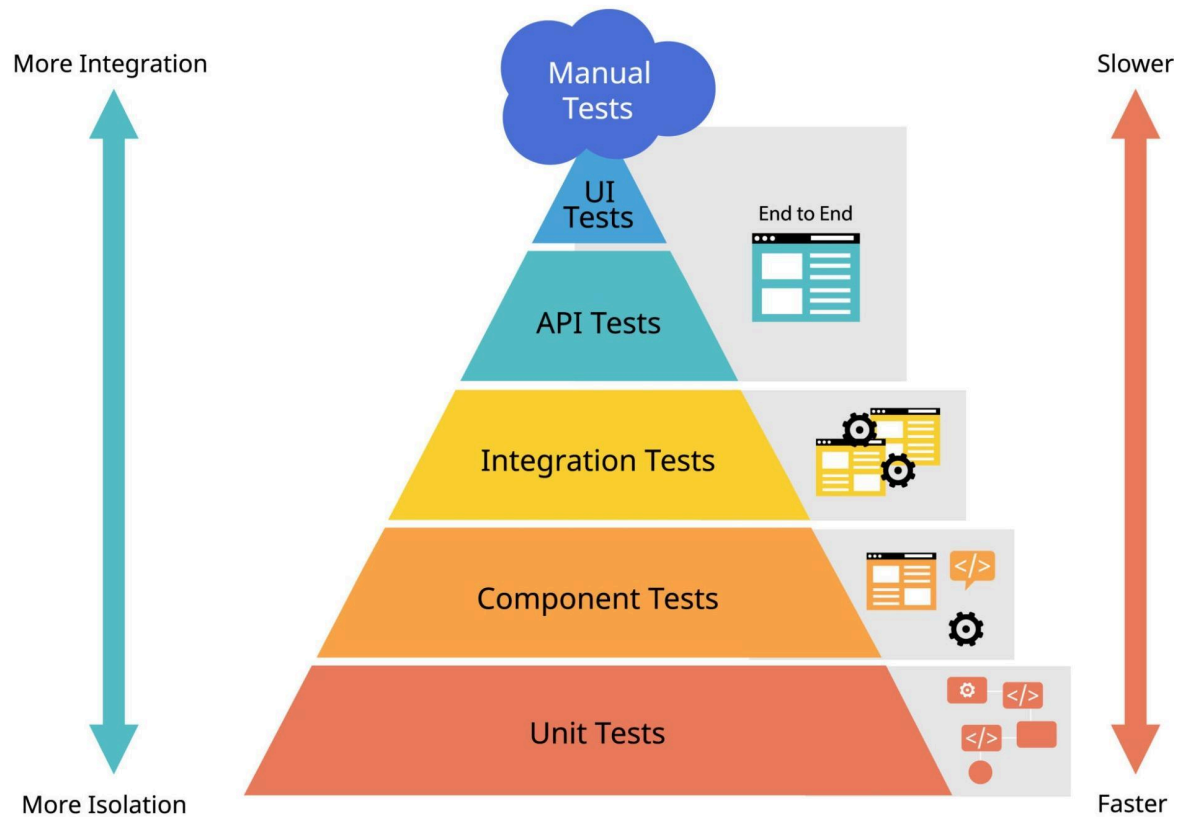
- **Método:** GET
- **Ruta:** /socios
- **Respuesta:** 200 OK con la lista de todos los socios registrados.

Módulo 2: Probando la Lógica de Negocio (Unit Testing)

Ahora que entendemos que el ServicioSocios es el cerebro que contiene las reglas de negocio, vamos a asegurarnos de que funciona correctamente **antes** de conectarlo a Javalin o a una base de datos.

2.1. Configuración del Test (Arrange)

Vamos a crear el archivo ServicioSociosTest.java.



Getty Images

Para probar el Servicio, necesitamos simular su dependencia: el Repositorio. No queremos usar el repositorio real porque no queremos depender de datos reales ahora mismo.

```
package gym;
```

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

```
class ServicioSociosTest {
```

```
    private ServicioSocios servicio;
```

```
    @BeforeEach
```

```
    void prepararEscenario() {
```

```
        // ARRANGE (Organizar)
```

```
        // Simulamos un entorno limpio antes de cada test.
```

```

        // Usamos una implementación en memoria del repositorio para facilitar las pruebas.
        RepositorioSocios repo = new RepositorioSocios();
        servicio = new ServicioSocios(repo);
    }

    // Aquí escribiremos los tests...
}

```

2.2. Probando el "Camino Feliz"

El primer test verifica que, si cumplimos las reglas, el sistema funciona.

```

@Test
void registrar_debe_guardar_socio_cuando_datos_son_validos() {
    // 1. ARRANGE
    String nombre = "Ana";
    String email = "ana@gym.com";

    // 2. ACT (La acción a probar)
    Socio resultado = servicio.registrar(nombre, email);

    // 3. ASSERT (La verificación)
    // Verificamos que no sea nulo
    assertNotNull(resultado, "El socio devuelto no debe ser null");

    // Verificamos que los datos sean correctos
    assertEquals("Ana", resultado.getNombre());
    assertEquals("ana@gym.com", resultado.getEmail());

    // Verificamos que el sistema le asignó un ID
    assertNotNull(resultado.getId(), "El socio debe tener un ID asignado");
}

```

2.3. Probando las Reglas de Negocio (Casos de Error)

Ahora probamos que el sistema es robusto y rechaza datos inválidos.

Caso: Email Duplicado

```

@Test
void registrar_debe_lanzar_error_si_email_ya_existe() {
    // Arrange: Primero insertamos un socio válido
}

```

```

servicio.registrar("Pepe", "pepe@gym.com");

// Act & Assert: Intentamos insertar otro con el MISMO email
// assertThrows verifica que el código lance la excepción esperada
Exception excepcion = assertThrows(IllegalArgumentException.class, () -> {
    servicio.registrar("Juan", "pepe@gym.com");
});

// Verificamos que el mensaje de error sea el esperado
assertEquals("Ya existe un socio con ese email", excepcion.getMessage());
}

```

Caso: Datos Inválidos

```

@Test
void registrar_debe_lanzar_error_si_nombre_vacio() {
    assertThrows(IllegalArgumentException.class, () -> {
        servicio.registrar("", "email@valido.com");
    });
}

@Test
void registrar_debe_lanzar_error_si_email_invalido() {
    assertThrows(IllegalArgumentException.class, () -> {
        servicio.registrar("Ana", "email-sin-arroba");
    });
}

```

Módulo 3: Introducción a los Mocks (Aislamiento Total)

En los tests anteriores usamos `new RepositorioSocios()`. Esto funciona porque nuestro repositorio es una lista en memoria. Pero, **¿y si fuera una base de datos real en la nube?** Los tests serían lentos y frágiles.

Para hacer un test unitario profesional, usamos **Mocks** (Simulacros). Un Mock es un objeto mentiroso: nosotros le decimos qué responder.

Vamos a refactorizar el test usando la librería **Mockito**.

3.1. Configurando Mockito

```
package gym;
```

```

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*; // Importante para usar 'when' y 'verify'

@ExtendWith(MockitoExtension.class) // Activamos Mockito
class ServicioSociosTestConMocks {

    @Mock // Crea un "doble" del repositorio (vacío, tonto)
    private RepositorioSocios repoMock;

    @InjectMocks // Crea el servicio real e inyecta el mock dentro
    private ServicioSocios servicio;

    @Test
    void registrar_debe_fallar_si_email_existe_mock() {
        // 1. ENSEÑAR AL MOCK (Stubbing)
        // "Oye Repositorio falso, si te preguntan por 'repe@gym.com',
        // responde que YA encontraste a alguien."
        Socio existente = new Socio("id1", "Old", "repe@gym.com");
        when(repoMock.buscarPorEmail("repe@gym.com")).thenReturn(existente);

        // 2. EJECUTAR
        assertThrows(IllegalArgumentException.class, () -> {
            servicio.registrar("New", "repe@gym.com");
        });

        // 3. VERIFICAR COMPORTAMIENTO
        // Nos aseguramos de que el servicio NUNCA llamó a guardar()
        // porque debió fallar antes.
        verify(repoMock, never()).guardar(any());
    }
}

```

Módulo 4: Probando la Capa Web (Javalin)

Finalmente, llegamos a la capa exterior: El Controlador.

El Problema: ¿Cómo probamos que POST /socios devuelve un código 201 sin levantar el servidor entero?

La Solución: Simulamos el objeto Context de Javalin.

El Context es el objeto que Javalin usa para darte la petición (body) y dejarte escribir la respuesta (json, status). Si lo mockeamos, podemos probar el controlador como si fuera una clase normal.

```
package gym;

import io.javalin.http.Context;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class SociosControllerTest {

    @Mock
    private ServicioSocios servicioMock; // Mockeamos el servicio

    @Mock
    private Context ctxMock; // ¡Mockeamos a Javalin!

    @InjectMocks
    private SociosController controller;

    @Test
    void crear_debe_retornar_201_y_json() {
        // --- 1. PREPARACIÓN (Arrange) ---

        // Simulamos el JSON que llega en la petición
        SocioCrear body = new SocioCrear();
        body.nombre = "Laura";
        body.email = "laura@gym.com";
        when(ctxMock.bodyAsClass(SocioCrear.class)).thenReturn(body);

        // Simulamos que el servicio crea el socio correctamente
        Socio socioCreado = new Socio("id-123", "Laura", "laura@gym.com");
```

```
when(servicioMock.registrar("Laura", "laura@gym.com")).thenReturn(socioCreado);

// Permitimos encadenar llamadas (ctx.status(...).json(...))
when(ctxMock.status(201)).thenReturn(ctxMock);

// --- 2. EJECUCIÓN (Act) ---
controller.crear(ctxMock);

// --- 3. VERIFICACIÓN (Assert) ---

// Verificamos que se llamó al servicio
verify(servicioMock).registrar("Laura", "laura@gym.com");

// Verificamos que se respondió con código 201
verify(ctxMock).status(201);

// Verificamos que se devolvió el JSON correcto
verify(ctxMock).json(socioCreado);
}
}
```

Resumen Final

Has aprendido a:

1. **Entender la Arquitectura:** Repositorio (Datos) < Servicio (Negocio) < Controlador (Web).
2. **Definir la API:** Establecer un contrato claro de entradas y salidas.
3. **Unit Testing:** Probar la lógica aislada.
4. **Mocking:** Simular dependencias (BD y Web) para tests rápidos y fiables.